

Why Choose Embabel Over Spring AI or LangGraph?

A Detailed Technical Comparison for JVM Developers

Version: 0.3.5-SNAPSHOT

Last Updated: March 2026

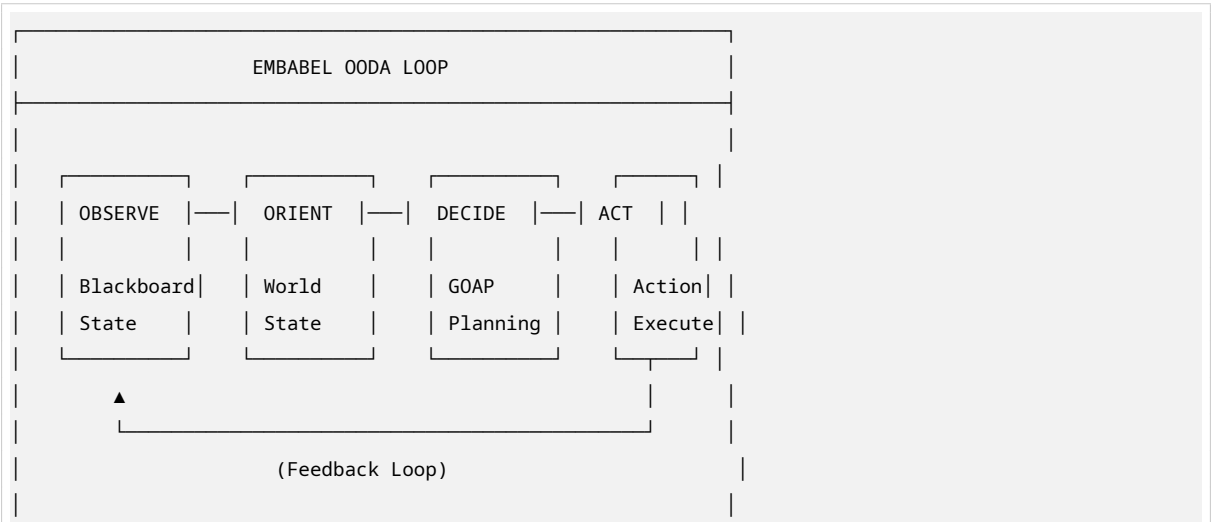
Executive Summary

Criteria	Embabel	Spring AI	LangGraph
Planning Capability	✅ GOAP + Supervisor + Utility	❌ None (workflows only)	❌ LLM-only orchestration
Type Safety	✅ Compile-time strong typing	⚠️ Limited	❌ Runtime Python
JVM Native	✅ Kotlin/Java first-class	✅ Java	❌ Python/JS only
Enterprise Integration	✅ Spring-native	✅ Spring-native	❌ External
Multi-Model Mixing	✅ Built-in	⚠️ Manual	⚠️ Manual
Deterministic Planning	✅ GOAP algorithm	❌ No planning	❌ LLM-dependent
Testability	✅ First-class testing	⚠️ Limited	⚠️ Limited
Production Maturity	✅ Growing (147 examples)	✅ VMware-backed	✅ 21.5K mature

1. The Fundamental Difference: Planning vs. Workflows

1.1 The OODA Loop Foundation

Embabel is built on the **OODA loop** (Observe, Orient, Decide, Act) - a decision-making model developed by USAF Colonel John Boyd. This is fundamentally different from workflow-based approaches:



Key Insight: Unlike Spring AI's sequential workflows or LangGraph's explicit graphs, Embabel's GOAP planner can discover novel solutions by combining actions in ways you never explicitly programmed.

1.2 Embabel's Three Planning Strategies

Planner	Best For	Determinism	Description
GOAP	Business processes	✅ High	Goal-Oriented Action Planning - AI algorithm from gaming
Utility	Exploration, event-driven	✅ High	Selects highest-value action at each step
Supervisor	Flexible multi-step	⚠️ Medium	LLM-orchestrated with type constraints

Spring AI: Only supports explicit workflows (no planning) **LangGraph:** Only supports explicit graphs (no AI planning)

2. Type Safety: The JVM Advantage

2.1 Embabel's Strong Typing

```
@Agent(description = "Type-safe agent")
public class TypedAgent {

    public record Person(String name, int age) {}
    public record Greeting(String message) {}

    @Action
    public Person extractPerson(UserInput input, OperationContext context) {
        // Compile-time type safety
        return context.ai()
            .withDefaultLlm()
            .createObject("Extract person from: " + input.getContent(), Person.class);
    }

    @AchievesGoal(description = "Greet the person")
    @Action
    public Greeting greetPerson(Person person) {
        return new Greeting("Hello, " + person.name());
    }
}
```

Benefits:

- ✅ Compile-time validation of data flow
- ✅ IDE autocomplete and refactoring support

-  Automatic JSON schema generation
-  Runtime type validation with automatic retry

2.2 Comparison: Type Safety

Aspect	Embabel	Spring AI	LangGraph
Compile-time checking	 Full	 Partial	 None (Python)
IDE support	 Excellent	 Good	 Limited
Refactoring	 Safe	 Manual	 Risky
Schema validation	 Automatic	 Manual	 Manual
Type inference	 Blackboard-based	 None	 None

3. The GOAP Advantage: True AI Planning

3.1 What is GOAP?

GOAP (Goal-Oriented Action Planning) is an AI algorithm originally developed for gaming (F.E.A.R., Deus Ex) that:

1. **Plans backward from goals** - Starts with the desired outcome
2. **Considers preconditions and effects** - Each action declares what it needs and produces
3. **Dynamically finds optimal paths** - Discovers novel action sequences
4. **Replans on failure** - Adapts when actions don't produce expected results

3.2 Embabel GOAP Example

```
@Agent(description = "Travel planning agent", planner = PlannerType.GOAP)
public class TravelPlanner {

    @Action(description = "Find flights")
    public FlightOptions findFlights(TravelRequest request, OperationContext context) {
        return context.ai()
            .withDefaultLlm()
            .createObject("Find flights for: " + request.destination(), FlightOptions.class);
    }

    @Action(description = "Book hotel", pre = {"spel:flightOptions != null"})
    public HotelBooking bookHotel(FlightOptions flightOptions, TravelRequest request) {
        // Only runs after flights are found
        return hotelService.book(request.destination(), flightOptions.arrivalDate());
    }

    @Action(description = "Rent car", pre = {"spel:hotelBooking != null"})
    public CarRental rentCar(HotelBooking hotel, TravelRequest request) {
        // Only runs after hotel is booked
        return carService.rent(hotel.location(), request.dates());
    }
}
```

```
@AchievesGoal(description = "Complete travel itinerary")
@Action
public TravelItinerary compileItinerary(
    FlightOptions flights,
    HotelBooking hotel,
    CarRental car) {
    return new TravelItinerary(flights, hotel, car);
}
```

Planner automatically determines:

```
TravelRequest → findFlights() → FlightOptions → bookHotel() → HotelBooking
→ rentCar() → CarRental → compileItinerary() → TravelItinerary (GOAL)
```

3.3 Why GOAP Matters

Benefit	Description
Novel Solutions	Can discover action combinations you never programmed
Dynamic Replanning	Adapts when actions fail or produce unexpected results
Parallel Execution	Identifies independent actions that can run in parallel
Composability	Add new actions without modifying existing code
Determinism	Same inputs produce same plans (unlike LLM-only)

Spring AI: Requires explicit workflow definition - no planning capability **LangGraph:** Requires explicit graph definition - no AI planning

4. Multi-Model Mixing: Cost Optimization

4.1 NVIDIA Research: SLMs are the Future

According to NVIDIA Research’s position paper “Small Language Models are the Future of Agentic AI”:

```
“Small language models (SLMs) are sufficiently powerful, inherently more suitable, and necessarily more economical for many invocations in agentic systems.”
```

Key Findings:

- SLMs are ideal for repetitive, specialized tasks
- Heterogeneous systems (multiple models) are optimal
- Significant cost savings with partial LLM-to-SLM shift

4.2 Embabel’s Built-in Model Mixing

```
@Agent(description = "Cost-optimized agent")
public class CostOptimizedAgent {

    @Action
```

```

public Analysis analyzeData(RawData data, OperationContext context) {
    // Use cheap model for simple extraction
    return context.ai()
        .withLlm(OpenAiModels.GPT_4O_MINI) // Cheapest
        .createObject("Analyze: " + data, Analysis.class);
}

@Action
public Report generateReport(Analysis analysis, OperationContext context) {
    // Use powerful model for complex synthesis
    return context.ai()
        .withLlm(AnthropicModels.CLAUDE_37_SONNET) // Best quality
        .createObject("Generate report from: " + analysis, Report.class);
}

@AchievesGoal(description = "Complete analysis and reporting")
@Action
public FinalOutput finalize(Report report, OperationContext context) {
    // Use local model for privacy-sensitive finalization
    return context.ai()
        .withLlm(OllamaModels.LLAMA_3_3) // Local, private
        .createObject("Finalize: " + report, FinalOutput.class);
}
}

```

4.3 Model Configuration

```

embabel:
  models:
    default-llm: gpt-4o-mini
    default-embedding-model: text-embedding-3-small
  llms:
    cheapest: gpt-4o-mini
    best: gpt-4o
    reasoning: o1-preview
    local: llama3.3

```

Spring AI: Manual provider switching, no built-in mixing **LangGraph:** Manual model selection, no optimization guidance

5. Enterprise Integration: Spring-Native

5.1 First-Class Spring Support

```

@SpringBootApplication
public class MyAgentApplication {

    @Bean
    public MyService myService() {
        return new MyService();
    }
}

```

```

    }
}

@Agent(description = "Spring-integrated agent")
public class SpringAgent {
    private final MyService myService;

    public SpringAgent(MyService myService) {
        this.myService = myService;
    }

    @Action
    public Result process(Input input, OperationContext context) {
        // Use Spring-managed services
        var data = myService.fetchData(input);
        return context.ai()
            .withDefaultLlm()
            .createObject("Process: " + data, Result.class);
    }
}

```

5.2 Spring AI vs Embabel

Feature	Spring AI	Embabel
Spring Boot integration	✓ Native	✓ Native
Dependency injection	✓ Full	✓ Full
Auto-configuration	✓ Yes	✓ Yes
Planning capability	✗ No	✓ GOAP
Agent abstraction	✗ Low-level	✓ High-level
Type-safe prompts	⚠ Limited	✓ Full
Testing support	⚠ Basic	✓ Comprehensive

Rod Johnson (Spring & Embabel founder):

“Embabel is a much higher level than LangChain4j and Spring AI. It builds on that layer, choosing Spring AI due to its affinity with the Spring ecosystem, but it offers a higher level of abstraction and introduces original ideas.”

6. Testing: First-Class Citizen

6.1 Unit Testing with FakeOperationContext

```

class MyAgentTest {

    @Test
    void testAgent() {
        var context = FakeOperationContext.create();
        var promptRunner = (FakePromptRunner) context.promptRunner();
    }
}

```

```

// Set up expected LLM response
context.expectResponse(new ExpectedOutput("test result"));

// Execute action
var agent = new MyAgent();
var result = agent.process(new Input("test"), context);

// Verify LLM was called correctly
assertEquals("test", promptRunner.getLlmInvocations().get(0).getPrompt());
assertEquals(0.7, promptRunner.getLlmInvocations().get(0)
    .getInteraction().getLlm().getTemperature(), 0.01);
}
}

```

6.2 Integration Testing

```

@SpringBootTest
class MyAgentIntegrationTest extends EmbabelMockitoIntegrationTest {

    @Autowired
    private AgentPlatform agentPlatform;

    @Test
    void testCompleteWorkflow() {
        // Mock LLM responses
        whenCreateObject(prompt -> prompt.contains("Process"), Output.class)
            .thenReturn(new Output("mocked result"));

        // Execute agent
        var invocation = AgentInvocation.create(agentPlatform, Output.class);
        var result = invocation.invoke(new Input("test"));

        // Verify
        assertNotNull(result);
        verifyCreateObjectMatching(
            prompt -> prompt.contains("Process"),
            Output.class,
            llm -> llm.getLlm().getTemperature() == 0.7
        );
    }
}

```

6.3 Testing Comparison

Aspect	Embabel	Spring AI	LangGraph
Unit testing	✅ FakeOperationContext	⚠️ Manual mocking	⚠️ Manual mocking

Table 6 – continued

Aspect	Embabel	Spring AI	LangGraph
Integration testing	✅ EmbabelMockitoIntegrationTest	⚠️ Limited	⚠️ LangSmith only
LLM mocking	✅ Built-in	❌ Manual	⚠️ External
Prompt verification	✅ Automatic	❌ Manual	⚠️ External
Test isolation	✅ Blackboard reset	⚠️ Manual	⚠️ Manual

7. Tool Integration: Framework-Agnostic

7.1 Creating Tools

```
public class MathTools {

    @LlmTool(description = "Add two numbers together")
    public double add(
        @LlmTool.Param(description = "First number") double a,
        @LlmTool.Param(description = "Second number") double b
    ) {
        return a + b;
    }

    @LlmTool(description = "Calculate square root")
    public double sqrt(
        @LlmTool.Param(description = "Number") double value
    ) {
        return Math.sqrt(value);
    }
}

// Use in agent
@Action
public Result calculate(UserInput input, OperationContext context) {
    return context.ai()
        .withDefaultLlm()
        .withToolObject(new MathTools())
        .createObject("Calculate: " + input.getContent(), Result.class);
}
```

7.2 MCP (Model Context Protocol) Support

```
spring:
  ai:
    mcp:
      client:
        type: SYNC
        stdio:
```



```

connections:
  brave-search-mcp:
    command: docker
    args: [run, -i, --rm, -e, BRAVE_API_KEY, mcp/brave-search]
    env:
      BRAVE_API_KEY: ${BRAVE_API_KEY}

```

8. Human-in-the-Loop (HITL)

8.1 WaitFor Pattern

```

@State
record ReviewState(UserInput input, Draft draft) implements Stage {

    @Action
    HumanFeedback getFeedback(OperationContext context) {
        return WaitFor.formSubmission("""
            Please review this draft:
            %s
            """.formatted(draft.content()), HumanFeedback.class);
    }

    @Action(clearBlackboard = true)
    Stage processFeedback(HumanFeedback feedback, Ai ai) {
        if (feedback.isApproved()) {
            return new Done(input, draft);
        } else {
            return new ReviseState(input, draft, feedback.comments());
        }
    }
}

```

8.2 HITL Comparison

Feature	Embabel	Spring AI	LangGraph
Native interrupt	✔ WaitFor	✘ Custom	✔ interrupt()
State persistence	✔ Automatic	⚠ Manual	✔ Checkpoint
Form submission	✔ Built-in	✘ Custom	⚠ External
Approval gates	✔ Configurable	✘ Custom	✔ Configurable

9. Real-World Use Cases

9.1 When to Choose Embabel

Scenario	Why Embabel
Enterprise Java/Spring	Native integration, familiar patterns
Complex multi-step workflows	GOAP planning discovers optimal paths
Cost-sensitive applications	Built-in multi-model mixing
Regulated industries	Deterministic planning, audit trails
Type safety critical	Compile-time validation
Existing JVM codebase	Leverage existing services and libraries

9.2 When to Choose Alternatives

Scenario	Better Alternative
Python-only team	LangGraph
Simple sequential workflows	Spring AI
Maximum community support	LangGraph (21.5K stars)
Rapid prototyping	LangGraph or CrewAI

10. Migration Path

10.1 From CrewAI

CrewAI Pattern:

```
researcher = Agent(  
    role='Research Analyst',  
    goal='Analyze market trends',  
    tools=[search_tool]  
)  
  
task = Task(  
    description='Research AI market',  
    agent=researcher  
)
```

Embabel Equivalent:

```
@Agent(description = "Research analyst agent")  
public class ResearchAgent {  
  
    @Action  
    public MarketAnalysis analyze(UserInput input, OperationContext context) {  
        return context.ai()  
            .withDefaultLlm()  
            .withToolGroup("web")  
            .createObject("Analyze market: " + input.getContent(), MarketAnalysis.class);  
    }  
  
    @AchievesGoal(description = "Complete market analysis")
```

```

    @Action
    public Report generateReport(MarketAnalysis analysis, OperationContext context) {
        return context.ai()
            .withDefaultLlm()
            .createObject("Generate report from: " + analysis, Report.class);
    }
}

```

Key Advantages:

-  Type safety - compile-time validation
-  Spring integration - leverage enterprise infrastructure
-  Automatic planning - GOAP handles task sequencing
-  Better testing - comprehensive test support

10.2 From LangGraph

LangGraph Pattern:

```

workflow = StateGraph(GraphState)
workflow.add_node("agent", call_model)
workflow.add_node("action", call_tool)
workflow.add_edge(START, "agent")
workflow.add_conditional_edges("agent", should_continue)

```

Embabel Equivalent:

```

@Agent(description = "Agent with tool usage")
public class ToolUsingAgent {

    @Action
    public AgentResponse process(UserInput input, OperationContext context) {
        return context.ai()
            .withDefaultLlm()
            .withToolObject(new MyTools())
            .createObject("Process: " + input.getContent(), AgentResponse.class);
    }

    @AchievesGoal(description = "Complete task")
    @Action
    public FinalOutput finalize(AgentResponse response) {
        return new FinalOutput(response);
    }
}

```

Key Advantages:

-  Type-safe - no runtime errors from state mismatches
-  Automatic planning - no manual edge definition
-  Spring ecosystem - enterprise integration

-  Better IDE support - refactoring, autocomplete

11. Summary: The Embabel Advantage

11.1 Core Differentiators

Feature	Embabel	Competitors
GOAP Planning	 Native	 None
Type Safety	 Compile-time	 Runtime
Spring Integration	 First-class	 External
Multi-Model Mixing	 Built-in	 Manual
Testing	 Comprehensive	 Limited
JVM Native	 Kotlin/Java	 Python

11.2 The Bottom Line

Choose Embabel if:

- You're building on the JVM (Java/Kotlin)
- You need true AI planning (not just workflows)
- Type safety and compile-time checking matter
- You're in an enterprise Spring environment
- Cost optimization through model mixing is important
- Testability is a priority

Choose LangGraph if:

- You're already in Python
- You need maximum community support
- Explicit graph control is required
- Rapid prototyping is the priority

Choose Spring AI if:

- You need simple, predictable workflows
- No planning capability is acceptable
- You're already invested in Spring ecosystem
- Agent abstraction isn't needed

12. Resources

- **Documentation:** docs.embabel.com
- **Examples:** github.com/embabel/embabel-agent-examples

- **Java Template:** github.com/embabel/java-agent-template
- **Kotlin Template:** github.com/embabel/kotlin-agent-template
- **NVIDIA SLM Research:** research.nvidia.com/labs/lpr/slm-agents
- **OODA Loop:** wikipedia.org/wiki/OODA_loop

From the creator of Spring Framework - bringing enterprise-grade agent development to the JVM ecosystem.

2024-2026 Embabel Pty, Ltd